Roll No: _____ **Exp. No:** 4a **Date:** _____

Experiment 4a: CPU Scheduling

CPU Scheduling: CPU scheduling is the process of selecting one of the processes in the ready queue to be executed by the CPU. It is a crucial component of multiprogramming operating systems, where multiple processes are kept in memory, and the CPU must be shared among them efficiently.

Criteria for job scheduling: The performance of CPU scheduling algorithms is measured using several criteria:

Criterion	Description
CPU Utilization	Keep the CPU as busy as possible (ideally 100%).
Throughput	Number of processes completed per unit time.
Turnaround Time	Time taken from submission to completion of a process.
Waiting Time	Time a process spends in the ready queue (not running).
Response Time	Time from submission to the first response (for interactive systems).
Fairness	Ensures that every process gets a fair share of the CPU.

CPU scheduling algorithms are divided based on:

- **Preemptive:** The CPU can be taken away from a running process.
  Examples: Round Robin, SRTF, Preemptive Priority
- **Non-Preemptive:** Once the CPU is given, the process runs until it finishes or blocks.
  Examples: FCFS, SJF, Non-preemptive Priority

1. First Come First Serve (FCFS) Scheduling

Aim: Write a program to demonstrate First Come First Serve (FCFS) job scheduling.

Description: This program implements FCFS CPU Scheduling, where processes are executed in the order of their arrival times.

- **Input Section:** The user provides the number of processes (n), and for each process: name, arrival time (AT), and burst time (BT).
- **Sorting:** Processes are sorted according to arrival time (AT) to ensure correct FCFS order.
- **Calculations:**
  - **Completion Time (CT):** The time when a process finishes execution.
    Formula: $CT[i] = CT[i-1] + BT[i]$
  - **Turnaround Time (TAT):** The total time taken from arrival to completion.
    Formula: $TAT[i] = CT[i] - AT[i]$

  - **Waiting Time (WT):** Time a process spends waiting in the ready queue.
    Formula: $WT[i] = TAT[i] - BT[i]$
    (If WT is negative, it's set to 0, meaning the process started immediately).

- **Output:** A table of processes with AT, BT, CT, TAT, and WT. Average TAT and average WT.

Algorithm:

1. Start
2. Input n (number of processes)
3. For each process: Input name, AT, BT
4. Sort processes by AT (bubble sort)
5. Initialize $sum = 0$
6. For each process i:
   $sum = sum + BT[i]$
   $CT[i] = sum$
7. For each process i:
   $TAT[i] = CT[i] - AT[i]$
   Add to $totalTAT$
8. For each process i:
   $WT[i] = TAT[i] - BT[i]$
   If $WT[i] < 0$ then $WT[i] = 0$
   Add to $totalWT$
9. Display process table (Ps, AT, BT, CT, TAT, WT)
10. Display avg TAT and avg WT
11. End

Roll No: _____ **Exp. No:** 4a **Date:** _____

Program:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, at[10], bt[10], wt[10], ct[10], tat[10], temp;
    float totaltat = 0, totalwt = 0;
    char names[10], tempc;

    printf("Enter no.of processors: ");
    scanf("%d", &n);

    for (int a = 0; a < n; a++) {
        printf("Processor name, at, bt: ");
        scanf(" %c %d %d", &names[a], &at[a], &bt[a]);
    }

    // Sort processes by Arrival Time
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                temp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = temp;

                temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;

                tempc = names[j];
                names[j] = names[j + 1];
                names[j + 1] = tempc;
            }
        }
    }

    int sum = 0;
    for (int k = 0; k < n; k++) {
        sum = sum + bt[k];
        ct[k] = sum;
    }

    for (int l = 0; l < n; l++) {
        tat[l] = ct[l] - at[l];
        totaltat = totaltat + tat[l];
    }

    for (int m = 0; m < n; m++) {
        wt[m] = tat[m] - bt[m];
        if (wt[m] < 0) {
            wt[m] = 0;
        }
        totalwt = totalwt + wt[m];
    }

    printf("\nPs\tAT\tBT\tCT\tTAT\tWT\n");
    for (int f = 0; f < n; f++) {
```

```
57          printf("%c\t%d\t%d\t%d\t%d\t%d\n", names[f], at[f], bt[f],
                ct[f], tat[f], wt[f]);
58      }
59
60      printf("Avg TAT=%.2fms\n", totaltat / n);
61      printf("Avg WT=%.2fms\n", totalwt / n);
62      return 0;
63  }
```

Output:

```
student@cselab2-10:~/160123733150$ gcc fcfs.c
student@cselab2-10:~/160123733150$ ./a.out
Enter no.of processors: 3
Processor name, at, bt: A 0 4
Processor name, at, bt: B 1 3
Processor name, at, bt: C 2 2

Ps      AT      BT      CT      TAT     WT
A       0       4       4       4       0
B       1       3       7       6       3
C       2       2       9       7       5
Avg TAT=5.67ms
Avg WT=2.67ms
```

2. Shortest Job First (SJF) Scheduling

Aim: Write a program to demonstrate Shortest Job First (SJF) scheduling.

Description: This program implements the Shortest Job First (SJF) non-preemptive CPU Scheduling algorithm assuming all processes arrive at time 0.

- **Input:** The number of processes, followed by each process's name and burst time.
- **Sorting:** Processes are sorted in ascending order of burst time to follow SJF rules.
- **Calculations:**
  - **Completion Time (CT):** Cumulative sum of burst times.
  - **Turnaround Time (TAT):** Since arrival time is zero, $TAT = CT$.
  - **Waiting Time (WT):** $WT = TAT - BT$ (negative waiting times are set to zero).
- **Output:** A table listing process details (BT, CT, TAT, WT) and the average turnaround and waiting times.

Algorithm:

1. Start
2. Input n (number of processes)
3. For each process: Input name, BT (burst time)
4. Sort processes by BT (ascending order) using bubble sort
5. Initialize $sum = 0$
6. For each process i:
   $sum+ = BT[i]$
   $CT[i] = sum$
7. For each process i:
   $TAT[i] = CT[i]$ (since AT = 0)
   Add to $totalTAT$
8. For each process i:
   $WT[i] = TAT[i] - BT[i]$
   If $WT[i] < 0$ then set $WT[i] = 0$
   Add to $totalWT$
9. Display table (Ps, BT, CT, TAT, WT)
10. Display average TAT and WT
11. End

Roll No: _____ **Exp. No:** 4a **Date:** _____

Program:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, bt[10], wt[10], ct[10], tat[10], temp;
    float totaltat = 0, totalwt = 0;
    char names[10], tempc;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input: process name and burst time only
    for (int a = 0; a < n; a++) {
        printf("Enter process name and burst time: ");
        scanf(" %c %d", &names[a], &bt[a]);
    }

    // Sorting by burst time (SJF non-preemptive)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (bt[j] > bt[j + 1]) {
                // Swap burst time
                temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;

                // Swap names accordingly
                tempc = names[j];
                names[j] = names[j + 1];
                names[j + 1] = tempc;
            }
        }
    }

    // Calculate Completion Time
    int sum = 0;
    for (int k = 0; k < n; k++) {
        sum += bt[k];
        ct[k] = sum;
    }

    // Turnaround Time = Completion Time (since arrival time = 0)
    for (int l = 0; l < n; l++) {
        tat[l] = ct[l];
        totaltat += tat[l];
    }

    // Waiting Time = TAT - BT
    for (int m = 0; m < n; m++) {
        wt[m] = tat[m] - bt[m];
        if (wt[m] < 0) {
            wt[m] = 0;
        }
        totalwt += wt[m];
    }

```

```
    // Output
    printf("\nPs\tBT\tCT\tTAT\tWT\n");
    for (int f = 0; f < n; f++) {
        printf("%c\t%d\t%d\t%d\t%d\n", names[f], bt[f], ct[f], tat[f],
            wt[f]);
    }

    printf("Avg TAT=%.2f ms\n", totaltat / n);
    printf("Avg WT=%.2f ms\n", totalwt / n);

    return 0;
}
```

Output:

```
student@cselab2-10:~/160123733150$ gcc sjfl.c
student@cselab2-10:~/160123733150$ ./a.out
Enter number of processes: 5
Enter process name and burst time: A 9
Enter process name and burst time: B 4
Enter process name and burst time: C 5
Enter process name and burst time: D 8
Enter process name and burst time: E 6

Ps      BT      CT      TAT     WT
B       4       4       4       0
C       5       9       9       4
E       6       15      15      9
D       8       23      23      15
A       9       32      32      23
Avg TAT=16.60 ms
Avg WT=10.20 ms
```